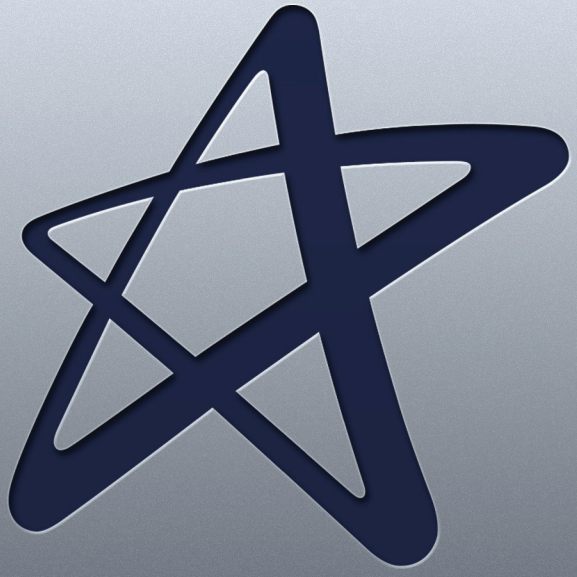
Pós-Graduação

Banco de Dados Open Source



Cruzeiro do Sul Virtual
Educação a Distância

Tipos de Bancos de Dados NoSQL

Responsável pelo Conteúdo:
Prof. Me. Luiz Carlos Reis

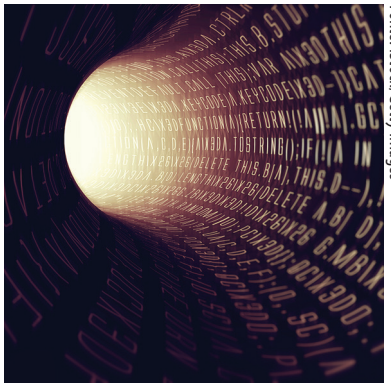
Revisão Textual:
Prof. Me. Luciano Vieira Francisco

Unidade

Tipos de Bancos de Dados NoSQL

Nesta unidade, trabalharemos os seguintes tópicos:

- Contextualização
- Introdução;
- Tipos de Bancos de Dados NoSQL;
- Banco de Dados Orientado a Chave-Valor;
- Bancos de Dados Orientados a Documentos;
- Bancos de Dados Orientados a Colunas;
- Bancos de Dados Orientados a Grafos;
- Conclusão.



Fonte: iStock/Getty Images

Objetivos

- Identificar as características principais de um banco de dados NoSQL;
- Estudar os quatro modelos de dados atualmente existentes, orientados a chave-valor, documentos, colunas e grafos.

Caro Aluno(a)!

Normalmente, com a correria do dia a dia, não nos organizamos e deixamos para o último momento o acesso ao estudo, o que implicará o não aprofundamento no material trabalhado ou, ainda, a perda dos prazos para o lançamento das atividades solicitadas.

Assim, organize seus estudos de maneira que entrem na sua rotina. Por exemplo, você poderá escolher um dia ao longo da semana ou um determinado horário todos ou alguns dias e determinar como o seu "momento do estudo".

No material de cada Unidade, há videoaulas e leituras indicadas, assim como sugestões de materiais complementares, elementos didáticos que ampliarão sua interpretação e auxiliarão o pleno entendimento dos temas abordados.

Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois estes ajudarão a verificar o quanto você absorveu do conteúdo, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e aprendizagem.

Bons Estudos!

Contextualização

Com a crescente popularização da internet, diversos novos dados foram surgindo e tratá-los foi se tornando gradualmente mais complexo e sua manutenção cada vez mais cara.

Bancos de dados relacionais escalam, mas quanto maior o tamanho, mais custoso se torna essa escalabilidade, seja pelo custo de novas máquinas, seja pelo aumento de especialistas nos bancos de dados utilizados.

Já os modelos não relacionais permitem uma escalabilidade mais barata e menos trabalhosa, pois não exigem máquinas extremamente poderosas e sua facilidade de manutenção permite que um número menor de profissionais seja necessário.

Assim, os bancos de dados NoSQL vão ficando mais populares entre as grandes empresas, pois reúnem as características de trabalharem com dados semiestruturados ou advindos de diversas origens – arquivos de log, sensores em máquinas e rodovias, websites, arquivos multimídia etc.

Portanto, estudaremos os modelos de tipos de banco de dados *open source* NoSQL.

Introdução

Pensando em solucionar diversos problemas relacionados à escalabilidade, performance e disponibilidade, projetistas do NoSQL promoveram uma alternativa de alto armazenamento com velocidade e grande disponibilidade, procurando se livrar de certas regras e estruturas que norteiam o modelo relacional.

Se por um lado havia um rompimento das regras do modelo relacional, por outro lado havia ganho de performance, flexibilizando os sistemas de banco de dados para as diversas características que são peculiares de cada empresa. Essa flexibilidade passou a se tornar fundamental para suprir os requisitos de alta escalabilidade, necessários para gerenciar grandes quantidades de dados, assim como para garantir alta disponibilidade, característica fundamental para as aplicações web 2.0.

O termo *NoSQL* foi originalmente utilizado em 1998, como o nome de um Banco de Dados (BD) não relacional de código aberto. Seu autor, Carlo Strozzi ([199-?]), alega que o movimento NoSQL "[...] é completamente distinto do modelo relacional e, portanto, deveria ser mais apropriadamente chamado 'NoREL' ou algo que produzisse o mesmo efeito".

Com a crescente popularização da internet, diversos novos dados foram surgindo, de modo que tratá-los foi se tornando gradualmente mais complexo e sua manutenção cada vez mais cara.

Em 2006, o artigo intitulado *Big-Table: a distributed storage system for structured data*, publicado pelo Google – e disponível em: https://goo.gl/icAMBD – trouxe novamente à tona o conceito *NoSQL*.

No início de 2009, o termo *NoSQL* foi reintroduzido por um funcionário do *Rackspace*, Eric Evans, quando Johan Oskarson, da *Last.fm,* queria organizar um evento para discutir bancos de dados *open source* distribuídos.

O nome era uma tentativa de descrever o surgimento de um número crescente de bancos de dados não relacionais e fazia uma referência ao esquema de atribuição de nomes dos bancos de dados relacionais mais populares do mercado, tais como MySQL, MS SQL, PostgreSQL etc.

A partir de então, os bancos de dados não relacionais passaram a ser conhecidos como *NoSQL*, e com a crescente popularização das redes sociais, a geração de conteúdo por dispositivos móveis, bem como o número cada vez maior de pessoas e dispositivos conectados, fez com que o trabalho de armazenamento de dados tendo por objetivo utilizá-los em ferramentas analíticas, começasse a esbarrar nas questões de escalabilidade e custos de manutenção desses dados.

Bancos de dados relacionais escalam, mas quanto maior o tamanho, mais custoso se torna essa escalabilidade, seja pelo custo de novas máquinas, seja pelo aumento de especialistas nos bancos de dados utilizados. Já os modelos não relacionais permitem uma

escalabilidade mais barata e menos trabalhosa, pois não exigem máquinas extremamente poderosas e sua facilidade de manutenção possibilita menor número de profissionais.

Assim, os *bancos de dados NoSQL* vão ficando mais populares entre as grandes empresas, pois reúnem as características de poder trabalhar com dados semiestruturados ou advindos de diversas origens – arquivos de log, sensores em máquinas e rodovias, websites, arquivos multimídia etc.

Tipos de Bancos de Dados NoSQL

A característica de esquema flexível é relacionada em como os dados são agrupados entre si. Assim, surgiu grande quantidade de bancos de dados não relacionais que trabalham de diferentes maneiras, de modo que podemos classificar os modelos existentes em NoSQL de acordo com a estrutura que os dados são armazenados. Atualmente, existem quatro modelos principais, orientados a chave-valor, documentos, colunas e a grafos.

Banco de Dados Orientado a Chave-Valor

De todos os modelos de bancos de dados NoSQL, o orientado a chave-valor é o que possui a estrutura mais simples, pois utiliza chaves como identificadores das informações gravadas em um campo assinalado como valor. Comumente, a chave é composta de um campo do tipo string; já o campo valor pode conter diferentes tipos de dados, sem necessitar de um esquema predefinido, tal como acontece em bancos de dados relacionais. Ademais, bancos de dados orientados a chave-valor são adequados para aplicações que realizam leituras frequentes.

Existem, atualmente, diversas opções de bancos de dados orientados a chave-valor. Embora cada um possua as suas próprias características, todas as opções disponíveis são criadas com foco em oferecer flexibilidade, desempenho e escalabilidade no gerenciamento dos dados. Por esse motivo, tal modelo de banco de dados pode ser uma solução ideal para resolver questões de lentidão para a leitura e/ou escrita de dados em grande variedade e volume.

Com a sua estrutura simples, é capaz de otimizar o desempenho da consulta e realizar operações com alta vazão. Embora a estrutura simples do banco de dados orientado a chave-valor ofereça benefícios, possui também algumas limitações, por exemplo, o conteúdo do campo valor é "opaco", não sendo possível fazer uma indexação com esse campo e uma consulta mais complexa (MARQUESONE, 2017). A seguinte Figura apresenta um exemplo que armazena informações com tais características.

A chave representa um campo como nome e idade e o valor corresponde aos campos preenchidos com Mara Rúbia e 43, ou seja, a instância para o campo correspondente. Este modelo é caracterizado pela sua facilidade ao ser implementado, permitindo que os dados sejam acessados rapidamente através da chave, aumentando também a disponibilidade do acesso aos dados. Para manipulá-los, utilizamos comandos simples, tais como *get()* e *set()*, que retornam e capturam valores. Ademais, um problema enfrentado por esse tipo de banco de dados é que não permite a recuperação de objetos por meio de consultas mais complexas.

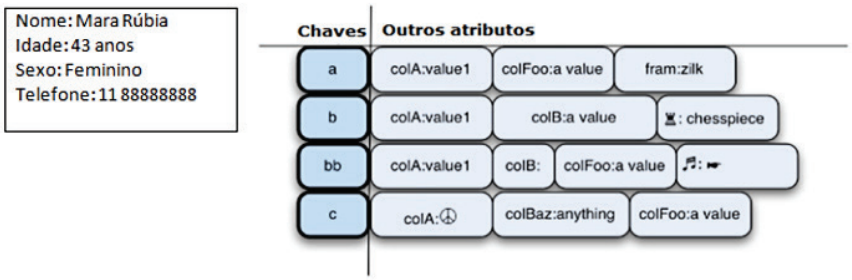


Figura 1 – Exemplo de armazenagem de dados em um banco de dados NoSQL do tipo chave/valor

A chave é o atributo único pelo qual todas as pesquisas são realizadas, pois nessa modalidade não é possível executar busca dentro do valor, dado que tal característica torna esse tipo de armazenamento escalar – com relação a input e output na base de dados –, podendo trabalhar uma boa quantidade de dados em memória – o que também é a sua principal limitação.

Seu uso é indicado no armazenamento de informação em sessão, em perfis de usuário e preferências, além de dados no carrinho de compras. Contudo, não é recomendado para relacionamento entre os dados, transações com múltiplas operações, consultas por dados, atributos e operações em conjunto.

Seguem exemplos de bancos de dados NoSQL *open source* com características de chave-valor:

- *DynamoDB – Amazon* –, disponível em: https://goo.gl/FUWb8X;
- *Redis*, disponível em: https://goo.gl/td48x2;
- *Riak*, disponível em: https://goo.gl/ryg8Q5;
- *Memcached*, disponível em: https://goo.gl/YSVG2K.

Bancos de Dados Orientados a Documentos

Considerado como extensão do banco de dados orientado a chave-valor, este tipo de banco de dados é provavelmente a categoria NoSQL mais popular na atualidade. Oferece meios de criação de índices sobre os valores dos dados armazenados, enriquecendo as possibilidades de consultas, além de permitir a normalização de dados e muitos outros conceitos oriundos do banco de dados relacional, tais como a criação de joins e definição de esquemas rígidos. No entanto, tais recursos não são obrigatórios, como acontece nos bancos de dados relacionais.

Podemos definir documentos como estruturas flexíveis que podem ser obtidas por meio de dados semiestruturados, tal como o formato *Extensible Markup Language* (XML) e *JavaScript Object Notation* (Json). Os desenvolvedores de aplicação web têm certa preferência por essa modalidade, pois a troca de dados utilizando o padrão Json é nativa nesse tipo de aplicação.

Pense em um documento como uma linha de uma tabela, e um conjunto de documentos como a tabela com todos os registros; a diferença é que cada documento – ou seja, cada linha da tabela – pode conter variações em sua estrutura. Isso é possível pelo fato de você não precisar definir um esquema antes de adicionar os registros.

Diferente do modelo chave-valor onde se cria uma única tabela hash, o banco de dados orientado a documentos permite a elaboração de consultas e filtros sobre os valores armazenados – e não somente pelo campo-chave. Permite trabalhar com a replicação dos dados em um cluster e, assim, garantir que o dado ficará disponível mesmo com a ocorrência de falha em um dos servidores. Esse modelo é indicado para realizar o armazenamento de conteúdo de páginas web, na catalogação de documentos de uma empresa e no gerenciamento de inventário de um e-commerce, pois são aplicações que trabalham diretamente com coleções de documentos (MARQUESONE, 2017).

Na seguinte Figura temos o exemplo de um determinado documento que foi definido por assunto, autor, data, tags e mensagens. Caso queira implementar outro campo chamado *Minha opinião sobre o site:*, não haverá qualquer tipo de restrição ou complexidade de inseri-lo no banco de dados, sendo essa flexibilidade um ponto forte nesse tipo de modelo.

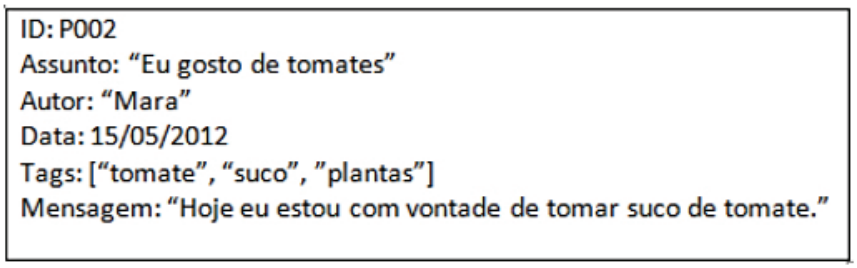


Figura 2 – Exemplo de arquivos para banco de dados NoSQL do tipo documento: modelo de documento

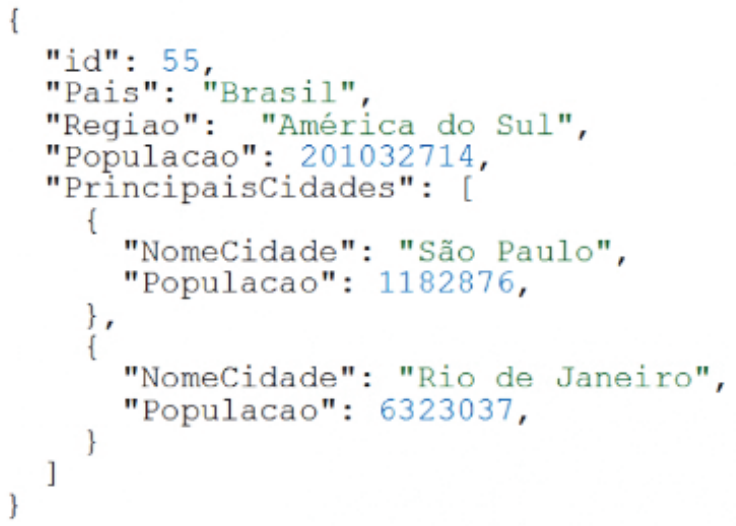


Figura 3 – Exemplo de arquivos para banco de dados NoSQL do tipo documento: estrutura Json

Entre as suas características, destaca-se o fácil acesso aos atributos internos do documento, contudo, consultas complexas que demandem dados de outros itens se tornam um pouco difíceis, pois todos os documentos envolvidos devem estar na mesma coleção; por isso, o seu uso não é indicado em tais circunstâncias.

Ademais, seguem exemplos de bancos de dados *open source* orientados a documentos:

- *Couchbase*, disponível em: https://goo.gl/mtNeGL;
- *CouchDB*, disponível em: https://goo.gl/EGdDHS;
- *MarkLogic*, disponível em: https://goo.gl/TqaptP;
- *MongoDB*, disponível em: https://goo.gl/JqLjuo.

Bancos de Dados Orientados a Colunas

De todos os modelos de armazenamento NoSQL, provavelmente o orientado a colunas é o mais complexo, afinal, este tipo de banco de dados foi criado para armazenar e processar grande quantidade de dados distribuídos em diversas máquinas.

Esse modelo também é considerado uma extensão do armazenamento orientado a chave-valor e possui conceitos similares ao do modelo relacional, como a criação de linhas e colunas. Entretanto, é preciso ficar atento(a), pois existem diferenças fundamentais no modo como essas estruturas são criadas.

Em tal cenário, como o banco de dados orientado a colunas se diferencia do banco de dados relacional?

Simples, esse tipo de banco busca resolver principalmente o problema de escalabilidade e flexibilidade no armazenamento de dados. No que se refere à flexibilidade, ao invés de definir antecipadamente as colunas necessárias para armazenar um registro, o responsável pela modelagem de dados estabelece o que é chamado de *famílias de colunas*.

As famílias de colunas são organizadas em grupos de itens de dados que são frequentemente usados em conjunto em uma aplicação. A partir das *famílias de colunas* definidas, o desenvolvedor possui a flexibilidade de inserir as colunas que considerar necessárias em cada registro armazenado, sem necessitar alterar a estrutura dos dados já inseridos.

Outra informação armazenada no banco de dados orientado a colunas é o timestamp de cada item gravado. Aqui existem as chaves, mas neste caso, apontam para colunas ou atributos múltiplos. Assim, os dados são indexados por uma tripla associação – coluna, linha e timestamp –, enquanto coluna e linha são identificadas por chaves e o timestamp permite diferenciar múltiplas versões de um mesmo dado.

Essa abordagem é empregada para prover o versionamento das colunas. Com essa estratégia de armazenamento por famílias de colunas, além de fornecer flexibilidade, esse modelo também permite grande escalabilidade. O registro de um item pode ter informações gravadas em diversas famílias de colunas, estas que podem estar armazenadas em diferentes servidores. Isso é possível pelo fato de os dados serem armazenados fisicamente em uma sequência orientada a colunas – e não por linhas.

Dessa forma, mesmo em um ambiente distribuído, com milhões de colunas, o banco de dados orientado a colunas pode disseminar as consultas em um grande número de nós de processamento sem realizar operações de join. Se a sua aplicação trabalha com volumes imensos de dados, se necessita de alto desempenho e disponibilidade na leitura e escrita dos dados, ou se carece de inclusão de campos dinâmicos e sua solução tolera eventuais inconsistências, provavelmente o banco de dados orientado a colunas é a solução mais adequada.

Por atender a tais necessidades, esse modelo é muito recorrido por aplicações de larga escala, tal como ocorre com o serviço de mensagens do *Facebook* (MARQUESONE, 2017).

A seguinte Figura modela o conceito de amigos, onde o primeiroNome e sobrenome são colunas pertencentes à família de colunas e denominada *nome*. Da mesma forma, as colunas endereço e cidade pertencem à família *local*. Ademais, é interessante observar que na linha 001 a pessoa Mara tem diversos endereços. Como a busca nesse tipo de banco de dados é atômica, mesmo que o interesse seja buscar o primeiroNome da linha 001, todas as colunas serão retornadas quando esta mesma linha for consultada. Permite ainda o particionamento de dados, oferecendo forte consistência.

Esse modelo de dados surgiu com o *BigTable* criado pelo Google; além do qual, temos também o *Cassandra,* que foi desenvolvido pelo *Facebook*.

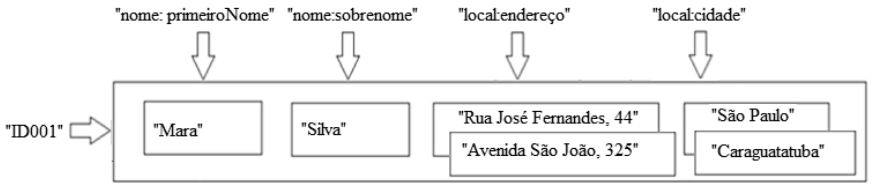


Figura 4 – Modelo de dados orientado a colunas

Os dados são agrupados como uma unidade celular por meio da key rown – chave – e seus dados. As colunas são dinâmicas, assim, caso o campo não seja preenchido, não será gerada uma coluna em branco na base de dados, como mostra o seguinte Quadro:

Quadro 1 – Modelo de dados orientados a colunas

Chave	Colunas			
ibsribas	name	email	adress	state
	Jessica	jb@email.com	onde judas perdeu as botas	CA
kvkvane	name	email	adress	state
	Ana Maria	kyka@email.com	terra do nunca	SO
vaminkes	name	email		
	Yasmin	ykesler@email.com		

Ademais, seguem exemplos de bancos de dados NoSQL *open source* orientados a colunas:

- *BigTable – Google –*, disponível em: https://goo.gl/68xDo6;
- *Accumulo*, disponível em: https://goo.gl/uPFYgb;
- *Cassandra – Facebook –*, disponível em: https://goo.gl/5rUW5P;
- *HBase*, disponível em: https://goo.gl/w7mfJ3;
- *Hypertable*, disponível em: https://goo.gl/yC38NR.

Bancos de Dados Orientados a Grafos

Entre os quatro tipos de armazenamento NoSQL apresentados, o orientado a grafos é provavelmente o mais especializado. Esse modelo possui três componentes básicos: nós – vértices dos grafos –, relacionamentos – arestas – e propriedades – conhecidas também como atributos.

Diferente dos outros modelos, em vez de os dados serem modelados utilizando um formato de linhas e colunas, possuem uma estrutura definida na teoria dos grafos, usando vértices e arestas para armazenar os dados dos itens coletados e os relacionamentos entre esses dados, respectivamente.

A utilização desse modelo é muito útil quando é necessário fazer consultas demasiadamente complexas, dado que o padrão orientado a grafos possui alta performance, permitindo bom desempenho nas aplicações que precisam traçar os caminhos existentes nos relacionamentos entre os dados como, por exemplo, as que necessitam identificar como um conjunto de amigos está conectado em uma rede, ou descobrir a melhor rota para se chegar a um local em menor tempo (MARQUESONE, 2017).

Para exemplificar, podemos analisar a seguinte Figura, a qual representa uma aplicação que mantém informações relativas à viagem, de modo que uma consulta pertinente seria: *Quais cidades foram visitadas anteriormente por pessoas que foram para Nova Iorque?* No modelo de banco de dados relacional tal pesquisa poderia se mostrar complexa, pois esse padrão não permite que os dados sejam representados de uma forma natural.

Prosseguindo, podemos citar o *Neo4j,* que é um banco de dados *open source*, baseado em grafos e desenvolvido em Java. Além de possuir suporte completo para *transactions*, trabalha também com nós e relacionamentos. Assim, ainda no exemplo da seguinte Figura, temos diversos indivíduos: João, Ricardo, Carolina, Maria, Fernando e Fábio, os quais representam nós do grafo e estão conectados a cidades que visitaram – então representadas pelas setas na cor azul – ou residiram – apontadas pelas setas na cor vermelha. Por exemplo: Ricardo viajou para Roma e Bruxelas e já residiu em Toronto e Paris. Assim, a partir de cada cidade, precisamos dos relacionamentos de entrada que também sejam do tipo *viajou* e, com isso, encontramos pessoas que viajaram para o mesmo lugar que Ricardo, neste caso, Carolina e Fernando.

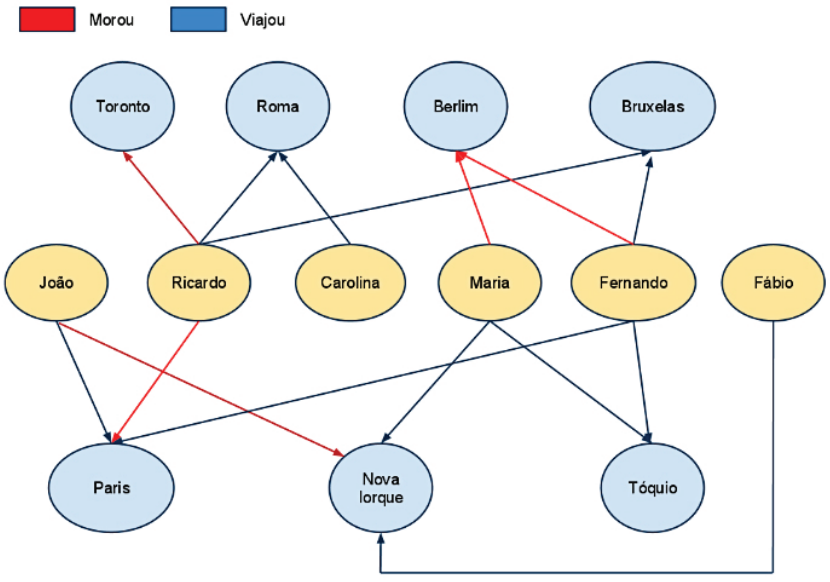


Figura 5 – Modelo de dados orientados a grafos

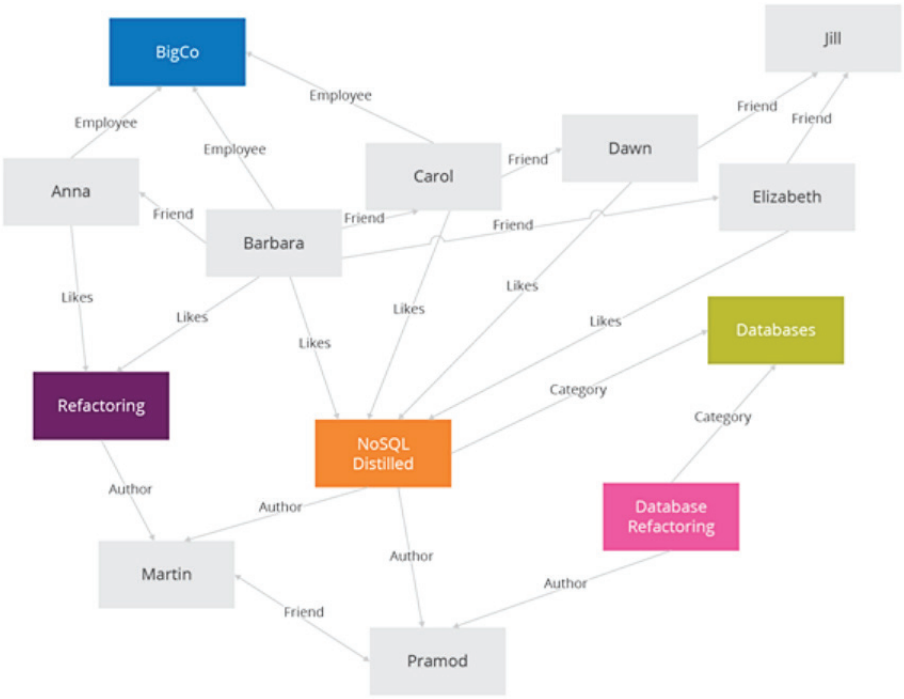


Figura 6 – Exemplo de armazenagem em banco de dados NoSQL do tipo grafo

Ademais, seguem exemplos de bancos de dados orientados a grafos *open source*:

- *AllegroGraph*, disponível em: https://goo.gl/Wiou5M;
- *ArangoDB*, disponível em: https://goo.gl/vAKxdU;
- *InfoGrid*, disponível em: https://goo.gl/wsEUWQ;
- *Neo4J*, disponível em: https://goo.gl/f73Hcs;
- *Titan*, disponível em: https://goo.gl/mwbkF3.

Conclusão

Por muitos anos, ao se projetar *softwares* na fase de definição da base de dados, já era inerente ao processo pensar nas tabelas e em suas relações. A única escolha a ser realizada era qual Sistema de Gerenciamento de Banco de Dados (SGBD) poderia se adequar melhor, ou seja, o foco era escolher o *software*, pois o modelo já se sabia que era relacional.

Mas isso mudou nos últimos anos devido à alteração de padrão de consumo e o grande crescimento do volume de dados, fazendo surgir o conceito NoSQL como grande alternativa quando nos referimos à escalabilidade e disponibilidade, fatores que se tornam imprescindíveis em algumas aplicações web.

Assim, analisando os quatro modelos descritos nesta oportunidade, é fundamental ressaltar que nenhum é superior aos demais. Na realidade, o que ocorre é que um modelo pode ser mais adequado para ser utilizado em certas situações. Por exemplo, para um BD de manipulação de dados que frequentemente serão escritos, mas não lidos – um contador de hits na web, por exemplo –, pode ser empregado um banco de dados orientado a documentos, tal como o *MongoDB*; já aplicativos que demandam alta disponibilidade, onde a minimização da atividade é essencial, podemos utilizar um modelo orientado a colunas, como o *Cassandra*; aplicações que exigem alto desempenho em consultas, com muitos agrupamentos, requerem um modelo orientado a grafos.

O importante é que no momento da criação do aplicativo, os desenvolvedores utilizem a melhor solução que se encaixe no perfil desejado. Empregar a solução adequada ao criar o BD significa diminuir os custos para a sua criação, assim como um banco eficiente no processamento de dados do ponto de vista de suas necessidades.

Para realizar a migração de um SGBD relacional para um banco de dados NoSQL, é preciso levar diversos fatores em consideração: a empresa em questão deve mensurar as diversas vantagens e desvantagens propostas por ambos os modelos, e estes critérios de comparação são dos mais diversos tipos, indo desde a escalabilidade do sistema, passando por avaliação sobre consistência de dados e quão importante é a disponibilidade do banco de dados para o sistema.

Acerca dos bancos de dados relacionais, sabemos da sua "experiência" no mercado, no qual é utilizado em larga escala. Além disso, sabemos da solidez de suas soluções que são mais maduras e experimentadas. Enquanto isso, os bancos de dados NoSQL ainda estão conquistando seu espaço no mercado e definindo os seus próprios padrões. Além disso, sabemos que para diversas organizações a consistência de dados se torna um fator determinante e as transações através dos SGBD relacionais correspondem a melhor alternativa para lidar com esse problema.

Por outro lado, temos a questão do grande volume de dados enfrentado por diversas empresas, assim como a necessidade de que esses sistemas estejam disponíveis aos seus usuários. Em tais situações, os bancos de dados NoSQL acrescentam diversos pontos positivos, primeiro pela sua possibilidade de escalonamento e simplicidade de seu modelo, onde não há esquemas pré-definidos; segundo, pela existência de grande distribuição de dados, oferecendo, assim, maior suporte de solicitações a esses dados para serem atingidos. Adicionalmente, é importante ressaltar que uma escalabilidade em alto grau se faz necessária a empresas que utilizam um banco de dados de grande porte e onde a disponibilidade é fator decisivo.

Mesmo com todas as mudanças que vêm ocorrendo devido ao fenômeno de big data, SGBD relacionais não deixarão a sua condição de usuais, pois muitas aplicações necessitam da segurança e confiabilidade dos dados que esse modelo proporciona. É comum que uma empresa tenha as suas bases de dados relacionais, executando em seus *softwares* tarefas cotidianas como, por exemplo, faturar pedidos. Esses mesmos dados também são enviados às aplicações de análises complexas.

É igualmente possível o uso de SGBD NoSQL em *softwares* do cotidiano, mas deve-se tomar um cuidado extra na modelagem de sua base de dados, pois ainda não há padronização dessa fase. Ademais, torna-se ainda necessário considerar um modelo de armazenamento NoSQL que favoreça a resolução do problema. Portanto, na fase de especificação do BD para o *software*, deve-se considerar, na característica do problema, quais requisitos são essenciais a serem sanados na taxa de crescimento dos dados para, apenas assim, definir a melhor estratégia de armazenamento a ser aplicada.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Leitura

BANCO de dados NoSQL: um novo paradigma

BANCO de dados NoSQL: um novo paradigma. SQL Magazine, n. 102, [20--].

https://goo.gl/onuWUf

Banco de dados NoSQL: uma visão geral

GROFFE, R. Banco de dados NoSQL: uma visão geral. iMasters, 30 set. 2016.

https://goo.gl/oSw9gB

Considerações sobre o banco de dados Apache Cassandra

PERERA, S. Considerações sobre o banco de dados Apache Cassandra. IBM DeveloperWorks, 20 jul. 2012.

https://goo.gl/xAMuP1

GraphDB series: o que é um banco de dados de grafos

SATO, P. M. GraphDB series: o que é um banco de dados de grafos. iMasters, 9 jan. 2014.

https://goo.gl/4gTBTm

Referências

AMAZON. **DynamoDB**. [20--]. Disponível em: <https://aws.amazon.com/pt/dynamodb/m/pt/dynamodb>. Acesso em: 23 jan. 2018.

APACHE. **Cassandra**. [20--a]. Disponível em: <http://cassandra.apache.org>. Acesso em: 23 jan. 2018.

APACHE. **CouchDB**. [20--b]. Disponível em: <http://couchdb.apache.org>. Acesso em: 23 jan. 2018.

CLOUD Bigtable. [20--]. Disponível em: <https://cloud.google.com/bigtable/?hl=pt-br>. Acesso em: 23 jan. 2018.

DECANDIA, G. et al. Dynamo: Amazon's highly available key-value store. **ACM Sigops Operating Systems Review**, v. 41, n. 6, p. 205-220, 2007.

ELMASRI, R.; NAVATHE, S. B.; DE OLIVEIRA MORAIS, R. **Sistemas de banco de dados**. [S.l.: s.n.], 2010.

HAN, J. et al. Survey on NoSQL database. In: PERVASIVE COMPUTING AND APPLICATIONS (ICPCA); INTERNATIONAL CONFERENCE ON (IEEE), 6., 2011. **Annales...** 2011. p. 363-366.

MARQUESONE. **Big Data**: técnicas e tecnologias para extração de valor de dados. São Paulo: Casa do Código, 2017.

MONGODB. [20--]. Disponível em: <https://www.mongodb.com>. Acesso em: 23 jan. 2018.

NEO4J. [20--]. Disponível em: <https://neo4j.com>. Acesso em: 23 jan. 2018.

PINTO, A. P. et al. Testes de performance utilizando o DB4O e MongoDB. **e-RAC**, v. 3, n. 1, 2013.

ROBINSON, I.; WEBBER, J.; EIFREM, E. **Graph databases**: new opportunities for connected data. [S.l.]: O'Reilly Media, 2015.

SADALAGE, P. J.; FOWLER, M. **NoSQL essencial**: um guia conciso para o mundo emergente da persistência poliglota. [S.l.]: Novatec, 2013.

VIEIRA, M. R. et al. Bancos de dados NoSQL: conceitos, ferramentas, linguagens e estudos de casos no contexto de Big Data. In: SIMPÓSIO BRASILEIRO DE BANCOS DE DADOS, 2012.

Cruzeiro do Sul Virtual
Educação a Distância

www.cruzeirodosulvirtual.com.br
Campus Liberdade
Rua Galvão Bueno, 868
CEP 01506-000
São Paulo - SP - Brasil
Tel: (55 11) 3385-3000



Cruzeiro do Sul
Educacional